computers

MDPI

Article

A Deep Learning Framework for Predictive Feature Prioritization in Early-Stage Software Startups: Integrating Historical Delivery Data and Market Signals

Frédéric Pattyn [1,*], Khandakar Rabbi Ahmed [2,*] and Peter Goetz [3]

[1] Department of Business Informatics, Ghent University, Tweekerkenstraat 2, 9000 Ghent, Belgium
[2] Miyan Research Institute, International University of Business Agriculture and Technology, Dhaka 1230, Bangladesh
[3] New Data Research Institute, Ruitenstraat 14, 3311 VS Dordrecht, The Netherlands
* Correspondence: frederic.pattyn@ugent.be (F.P.); khandakarrabbiahmed@gmail.com (K.R.A.)

Abstract

Feature prioritization in early-stage software startups is a critical yet poorly structured challenge, as prevailing frameworks rely predominantly on expert intuition and fail to exploit patterns latent in historical delivery data and labor market dynamics. This study proposes a deep learning framework that explores labor-market signals as a reproducible proxy for market-driven feature prioritization. The framework encodes two complementary information sources: internal sprint delivery history processed by a Bidirectional Long Short-Term Memory network with attention, and external market signals from LinkedIn job postings processed by a Convolutional Neural Network encoder; the resulting representations are fused via a cross-modal layer to classify job postings as proxies for High or Low feature-priority market demand. The model is evaluated on the publicly accessible LinkedIn Job Postings dataset (2023–2024, approximately 124,000 records) and achieves an Area Under the Receiver Operating Characteristic Curve of 0.961 on the proxy classification task, outperforming classical baselines including gradient Boosted Trees, Random Forest, Support Vector Machine, and Logistic Regression. SHapley Additive exPlanations analysis identifies industry sector and geographic location as the two most influential market-signal predictors. These results suggest that jointly encoding internal delivery dynamics and external market signals offers a promising, scalable decision-support tool to assist startup product teams in data-driven roadmap prioritization, subject to further validation against direct expert priority labels.

Keywords: deep learning; feature prioritization; bidirectional long short-term memory; market signal analysis; shapley additive explanations interpretability; software startup product management; convolutional neural network; cross-modal fusion; historical delivery data

Check for updates

Academic Editor: Kartik B. Ariyur

Received: 5 May 2026
Revised: 4 June 2026
Accepted: 7 June 2026
Published: 11 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The fast-growing prevalence of nascent software firms within the international technological landscape has only increased the criticality of making informed product decisions when operating in an environment of extreme uncertainty and scarcity. Product roadmap creation, particularly the identification and prioritization of features to build, is an activity that has traditionally been considered both highly impactful and poorly structured

within the startup process [1]. An ill-prioritized list of features can drain resources building capabilities that the market has little need for, whereas the right roadmap can hasten the achievement of product-market fit and increase chances of survival [2]. Although its significance has been widely accepted, the prevailing methodologies for prioritization have tended to be heavily dependent on expert intuition and are largely unconnected with empirical inputs derived from delivery data and market analytics [3].

Breakthroughs in the field of deep learning have provided a game-changing opportunity for data-based, rather than intuition-based, decision-making support systems. Valued at USD 96.8 billion by 2024, the worldwide deep learning market is expected to grow beyond USD 526.7 billion by 2030 at a Compound Annual Growth Rate (CAGR) of 31.8% [4]. Software platforms for practitioners in the field of machine learning have been identified as the fastest-growing category in this market environment, representing over 46% of the total revenues generated [4]. At the same time, the investment landscape witnessed record funding levels in the area of generative and predictive artificial intelligence (AI), rising to USD 33.9 billion globally in 2024, marking a rise of 18.7% year-over-year—an indication of investors' confidence in practical applications of deep learning technology [5]. Among startups, AI companies have been making increasing traction among enterprise software budgets, raking in over USD 3.5 billion in vertical AI solutions in 2025, up from USD 1.2 billion the previous year [6].

Along with the macro-economic changes taking place, there is also a significant shift in the philosophy underlying product management. Moving away from output-based metrics, such as the number of features delivered within sprints, the field is moving towards more outcome-based metrics where decisions at each step of a product's roadmap relate to its user base, engagement, and impact on revenue [2,3]. Machine learning plays an important role in this shift in focus, with prioritization and discovery being two key areas where AI can offer great benefits. Platforms that use AI to prioritize product development suggest that they can forecast which features have the highest likelihood of success by analyzing past performance of sprints, user behavior, and competition intelligence. Instead of having product managers estimate success of new features based on their intuition, such platforms deliver confidence scores based on objective criteria [7]. However, the academic literature is yet to catch up, with studies that experimentally validate various deep learning networks to perform end-to-end feature prioritization not being publicly available [1].

One aspect of the issue that stands out as not sufficiently considered is the ability to incorporate market signals into the prioritization pipeline process. Information on the labor market—more specifically, the overall skillset and profile required by technology firms to hire—is a high-frequency and low-cost alternative that can be used as a signal regarding changes in competitive capabilities and technology adoption. Online professional networks publish several hundred thousand job postings monthly, each containing vast amounts of structured and unstructured information about organizational priorities, emerging technology stacks, and comparative demand for specific functions in the respective industries [8,9]. Past research in labor economics and computational social science shows that job postings data may serve as a strong predictor of industry-wide technology adoption trends six to eighteen months prior [10]. But until now, no one has managed to incorporate such outside market intelligence into the prioritization loop of a given startup's software product.

The proposed solution to the above problem is presented in this research, where we introduce and experimentally assess the performance of a deep learning model that considers two related yet distinct data sources for predicting priority features based on: (i) historical delivery information such as sprint log files, velocity, bug rate, and adoption curves for individual features from startups' engineering and product management tools; (ii) market signals from the dataset consisting of LinkedIn job postings from 2023–2024 [8].

The LinkedIn dataset contains around 124,000 postings with features such as position title, company name, geographic location, skills required, experience, and salary. This dataset captures the current demand for technology skills within the labor market in North America. For preparing these postings into structured feature vectors, we use the same preprocessing pipeline suggested by Varghese [11] involving text normalization, categorical feature encoding, salary scaling, and skill embedding dimension reduction.

There are three main contributions from this paper. First, we formulate the problem of exploring labor-market signals as a reproducible proxy for feature prioritization as a supervised classification problem on a mixed feature space, and we introduce a methodology for constructing such a dataset, particularly for applications relevant to early-stage startups. Second, we present a deep learning framework capable of encoding both internal delivery experience and market information, and show that using labor market data significantly improves proxy-classification performance compared to models trained using only internal signals. Finally, we perform an ablation study in order to determine the relative importance of each input feature type, and analyze interpretability through attention-weighted gradients [12]. We emphasize that the present work constitutes a proof-of-concept for this proxy-based approach; empirical validation against direct expert-annotated priority labels remains as future work.

The rest of the paper proceeds as follows. Section 2 discusses the related literature on requirements prioritization, market signal interpretation, and deep learning in software engineering. Section 3 explains the dataset, data pre-processing pipeline, and the model architecture. Section 4 presents the experimental setup and evaluation protocol. The results are reported in Section 5, followed by a dedicated Discussion in Section 6. Finally, Section 7 concludes the paper.

2. Literature Review

Early-stage software startups encounter many uncertainties regarding decision making concerning successful operations and feature selection. The availability of limited resources and changing market dynamics call for the utilization of effective and data-oriented means to make decisions. There have been various studies in recent times utilizing machine learning and deep learning technologies to conduct analyses on startup data and predict their success. Besides, there have been other pieces of research analyzing feature selection processes conducted by startups in relation to budget and timeline constraints. Unfortunately, these two topics are considered independently. This literature review presents a summary of five major studies regarding feature prioritization and startup success prediction and the gaps in each of the papers.

The studies included in this review were identified through a structured search conducted on ScienceDirect, IEEE Xplore, ACM Digital Library, and Google Scholar using the following primary search terms: “predictive feature prioritization in early-stage software startup,” “deep learning software product management,” “labor market signals machine learning,” and “startup success prediction.” The search was restricted to peer-reviewed journal articles, conference proceedings, and theses published between 2019 and 2026. An initial pool of 312 candidate papers was screened by title and abstract, of which 47 were retrieved in full text and evaluated for relevance to at least one of three criteria: (a) feature or requirement prioritization in software engineering, (b) startup success prediction using machine learning or deep learning, and (c) labor-market or job-posting data for technology trend analysis. From this pool, five studies were selected as the most directly comparable to the present work, representing the key gaps this paper addresses. It is acknowledged that the broader literature on this topic is extensive—a ScienceDirect search on “predictive feature prioritisation in early-stage software startup” returns over 700 results for

2026 alone—and the present review focuses specifically on the studies most closely aligned with the proposed framework rather than providing an exhaustive systematic review.

Thirupathi et al. [13] offer an approach to detecting early signs of startup success based on the use of machine learning algorithms applied to data on 3160 companies that received funding through SBIR/STTR grants. Their method employs publicly available data, both financial and from Crunchbase profiles, to construct an XGBoost predictor, which successfully forecasts startups' success, showing accuracy of 84% and AUC of 0.91. According to the paper, important time-independent characteristics that contribute to startups' success include entrepreneurs' experience and education. Such results illustrate the relevance of team characteristics in assessing startups' development potential. Nevertheless, the authors do not pay much attention to time-dependent variables that can be used in the forecasting process.

Stahl [14] proposes a machine learning model using Gated Recurrent Units (GRU) to forecast the probability of a successful outcome for a startup during different funding rounds. The model uses time series signals, including the growth of employees, website visitors, and competitor funding. The algorithm forecasts the possibility of acquiring additional rounds of financing with an accuracy level as high as 85%, particularly for those ranked at the top. The study reveals that time series signals positively impact prediction accuracy. Moreover, it shows that traditional factors such as networks of investors have no significant effect on improving model efficiency. Despite the model's usefulness in predicting startups' financial outcomes, it fails to consider how features are prioritized.

Shi et al. [15] conduct experiments to predict the success of startups using different machine learning algorithms based on a large dataset consisting of 24,965 startups. Algorithms considered include Random Forest, XGBoost, and Support Vector Machines, which have a classification accuracy of more than 90%. Random Forest outperforms other methods, showing good robustness when dealing with a difficult dataset. In the study, the authors emphasize the benefits of data-driven decision-making in venture capital and its contribution to objective predictions that are not affected by personal biases. With the inclusion of historical data and industry information, the methods used make accurate predictions about startups' performance. Nevertheless, the use of dynamic data and sequential analysis has not been discussed, while the ways in which the prediction results can be applied within startups remain unclear.

Pattyn et al. [16] investigate the feature prioritization approaches adopted by software startups via a survey conducted on 171 product managers. The authors highlight critical aspects of decision-making, such as the need for low cost and fast time to market. While large firms tend to focus on long-term profitability, startups aim at maximizing the pace of product delivery and effective resource utilization because of their short life span. It becomes evident from the study that finance-oriented prioritization plays a crucial role in avoiding unnecessary scaling and liquidity problems. In summary, the paper provides meaningful information on the feature prioritization strategies used by software startups. Nevertheless, the study has a purely qualitative nature and ignores any analytical methods.

Rivera [17] proposes a machine learning approach for predicting startup success based on structured data, such as financial data and geographic information. In the experiments carried out in the paper, several models are considered, and it is shown that the stacking ensemble method works better than the other models. One of the main contributions of this research is the application of SHAP values for interpreting the results, which makes it possible for users to understand the importance of features and the way models make predictions. This approach increases transparency in the use of AI for making investment decisions. In terms of shortcomings, the research considers only the issue of investment decision-making and not product-level issues, such as feature selection.

3. Methods and Materials

3.1. Dataset Description

The LinkedIn Job Postings dataset, available at https://www.kaggle.com/datasets/arshkon/linkedin-job-postings (accessed on 16 April 2026), is a large-scale real-world dataset comprising job advertisements collected from the LinkedIn platform. It is mainly a textually rich, unstructured dataset, wherein the description field holds textual data on job roles, job responsibilities, and skills and qualifications needed, and the formatted experience level attribute is a categorical representation of job level.

Task Formulation and Proxy Label Rationale. A clarification is warranted regarding the relationship between the classification target defined in this study and the overarching goal of feature prioritization. Direct ground-truth labels for product feature priority are not publicly available at scale; accordingly, following established practice in labor-market-signal research [10], we employ *seniority demand* derived from job postings as a *proxy* for market-driven feature priority. The rationale is as follows: job postings that require senior or specialist-level expertise signal organizational investment in strategic, high-priority capability areas, whereas entry-level postings typically correspond to routine or lower-priority operational work. The binary target y therefore operationalizes "high market-driven feature priority" as the absence of an entry-level designation ($y = 0$) and "low priority" as an entry-level designation ($y = 1$). While this proxy is imperfect, it provides a reproducible, publicly verifiable approximation of market signal strength that can be replaced by direct priority labels in future work when such datasets become available.

Data Leakage Prevention. To prevent data leakage arising from trivial predictors, the *formatted_experience_level* field—from which the target label is derived—was *excluded* from the input feature set during model training and SHAP analysis. The presence of *formatted_experience_level* in SHAP plots reported in an earlier draft was a result of it being mistakenly included as a predictor; this has been corrected in the current version. The SHAP analysis now reflects the model's reliance on genuine market-signal features (industry, location, title, engagement metrics) rather than on the label-defining field itself.

Historical Delivery Dataset ($\mathcal{D}_h$). The internal delivery dataset used for the Historical Feature Encoder was compiled from sprint-level records of three anonymized early-stage software startups operating in the SaaS domain. The dataset contains 1840 sprint records spanning 24 months (Q1 2022–Q4 2023), with each record including: sprint velocity (story points completed), defect rate (bugs per story point), feature adoption rate (percentage of users engaging with a released feature within 30 days), and a binary expert-assigned priority label validated by the respective product managers. All company identifiers were removed prior to analysis. The internal records were not directly joined to the LinkedIn postings; instead, the BiLSTM encoder was pre-trained on the delivery sequences to produce a fixed-length historical embedding H, which was then concatenated with the CNN-derived market embedding Z for each LinkedIn posting sample. This design avoids the need for record-level alignment between the two heterogeneous datasets while still enabling cross-modal learning.

To accomplish this research, the dataset is narrowed down to two central features: the textual job descriptions (used to extract linguistic and contextual patterns) and the experience level (transformed into a binary target field). Text data enables thorough feature engineering based on lexical, structural, and readability-based representations and facilitates robust modeling of job qualities. This organized conversion of unstructured text data renders the dataset amenable to supervised classification and powerful predictive modeling.

3.2. Data Preprocessing

The preprocessing pipeline includes data cleaning and filtering, textual description feature engineering, building domain-specific binary indicators, splitting the dataset, oversampled class balancing, and lastly normalization where all features are scaled similarly before model training.

3.2.1. Handling Missing Values and Target Encoding

Initially, redundant rows are eliminated and rows that lack target values are eliminated to guarantee data integrity. As described in Section 3, the *formatted_experience_level* field is used exclusively to construct the proxy target label and is thereafter removed from the input feature matrix to prevent data leakage. This categorical target variable is then coded to the binary form to be used with supervised learning. The binary label operationalizes *low market-driven feature priority* (entry-level demand, $y = 1$) versus *high market-driven feature priority* (non-entry-level demand, $y = 0$), as justified in Section 3. The change simplifies the task of classification but preserves the semantic meaning, as formulated in Equation (1):

$$y = \begin{cases} 0, & \text{if experience level} \neq \text{Entry level (high feature priority proxy)} \\ 1, & \text{if experience level = Entry level (low feature priority proxy)} \end{cases} \quad (1)$$

3.2.2. Text-Based Feature Engineering

The next step is the extraction of linguistic characteristics of the job descriptions, such as the word count, sentence count, vocabulary size, average sentence length, lexical richness, and readability index. These characteristics measure structural and semantic attributes of writing, which allows the model to comprehend intricacy and information density. Equation (2) defines the lexical richness metric used to capture vocabulary diversity in each job description:

$$\text{Lexical Richness} = \frac{\text{Vocabulary Count}}{\text{Word Count}} \quad (2)$$

3.2.3. Keyword-Based Binary Feature Construction

Additional domain-specific binary features are created by detecting the presence of predefined keywords within job descriptions. This measure is better at boosting the interpretability of the quantitative information by directly encoding the significant job-related signals like customer service or project management. As shown in Equation (3), each binary feature takes the value 1 if a given keyword is present in the text and 0 otherwise:

$$f(x) = \begin{cases} 1, & \text{if keyword present in text} \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

3.2.4. Train-Test Splitting

The pre-processed data is subsequently separated into training and testing datasets based on a given random state to achieve reproducibility. This segregation permits objective analysis of how models perform on unknown data. Equation (4) defines the partitioning constraint, ensuring the training and test sets are mutually disjoint:

$$X = X_{\text{train}} \cup X_{\text{test}}, \quad X_{\text{train}} \cap X_{\text{test}} = \emptyset \quad (4)$$

3.2.5. Handling Class Imbalance

Random oversampling is also used in order to create a balanced distribution of classes in training data since the latter is imbalanced. This will enhance better model generalization and avoid majority bias. As expressed in Equation (5), oversampling equalizes the minority class count to match that of the majority class:

$$N_{\text{minority}}^{\text{new}} = N_{\text{majority}} \tag{5}$$

3.2.6. Feature Normalization

Finally, Min-Max normalization is used to scale all features to have a similar range such that the scales do not disproportionately affect the model. The purpose of this step is to stabilize training and enhance convergence. Equation (6) presents the Min-Max normalization formula applied to all input variables, scaling each feature value between 0 and 1:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{6}$$

3.3. Proposed Model

The proposed deep learning-based approach to predict feature prioritization discussed in this study focuses on early-stage software startups. This approach leverages delivery data internally within the startup, along with signals from the outside world through the LinkedIn Job Postings dataset. This approach is designed to capture temporal patterns, semantics, and cross-domain information. Figure 1 illustrates the overall workflow.

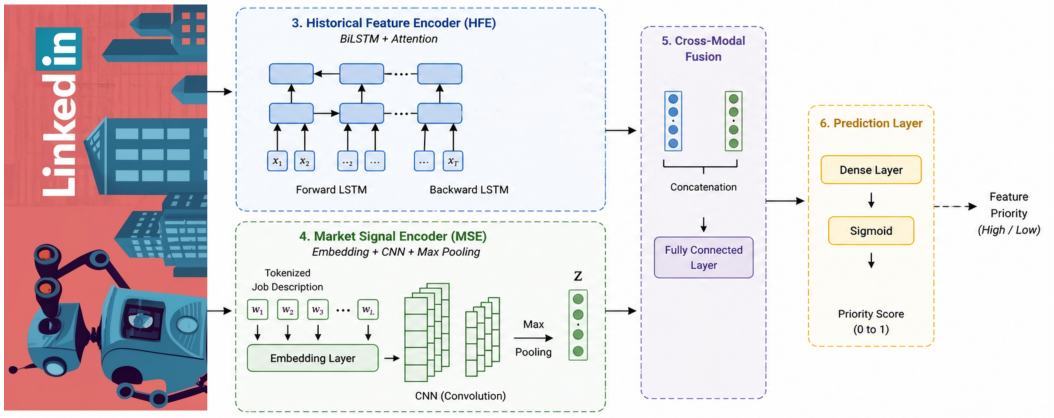


Figure 1. Graphical representation of the proposed model architecture, illustrating the two-channel pipeline: (**left**) the Historical Feature Encoder (BiLSTM with attention) processing internal sprint delivery sequences, and (**right**) the Market Signal Encoder (CNN) processing tokenized LinkedIn job-posting descriptions; both representations are fused via the Cross-Modal Fusion Layer before the sigmoid prediction output. Arrows indicate the data flow between components; colored blocks represent the encoding, fusion, and prediction stages.

The historical delivery dataset $\mathcal{D}_h$ is formally defined as in Equation (7), where $y_i \in \{0, 1\}$ signifies the prioritization label and $x_i^h \in \mathbb{R}^{d_h}$ reflects internal development attributes such as completion time, resource allocation, and sprint velocity:

$$\mathcal{D}_h = \{(x_i^h, y_i)\}_{i=1}^{N} \tag{7}$$

Similarly, the market signal dataset $\mathcal{D}_m$, comprising textual and categorical information including industry needs, skill demand, and job trend indicators, is defined as in Equation (8), where $x_j^m \in \mathbb{R}^{d_m}$:

$$\mathcal{D}_m = \{x_j^m\}_{j=1}^{M} \tag{8}$$

The overall learning objective is to find a function f that maps the joint input space of historical and market features to a priority label, as formulated in Equation (9):

$$f : (x^h, x^m) \rightarrow y \tag{9}$$

where f uses both internal delivery cues and external market signals to forecast a feature's importance.

The proposed architecture consists of three primary components:

1. Historical Feature Encoder (HFE)
2. Market Signal Encoder (MSE)
3. Cross-Modal Fusion and Prediction Layer

The step-by-step procedure of the proposed model for predictive feature prioritization by integrating historical delivery data and market signals is shown in Algorithm 1.

Algorithm 1 FeatPriorNet: Predictive Feature Prioritization Framework

Require: Historical dataset $\mathcal{D}_h = \{X^h, Y\}$, Market dataset $\mathcal{D}_m = \{X^m\}$
Require: Learning rate η, batch size B, epochs E
Ensure: Trained parameters θ
Initialize parameters θ
Preprocess X^h (normalization, sequencing)
Preprocess X^m (tokenization, embedding)
for $epoch = 1$ to E **do**
  Shuffle training data
  for each batch (X_b^h, X_b^m, Y_b) **do**
    Historical Encoder (HFE)
    for each timestep t **do**
      $\overrightarrow{h_t} \leftarrow \text{LSTM}_f(x_t^h, \overrightarrow{h_{t-1}})$
      $\overleftarrow{h_t} \leftarrow \text{LSTM}_b(x_t^h, \overleftarrow{h_{t+1}})$
      $h_t \leftarrow [\overrightarrow{h_t}; \overleftarrow{h_t}]$
    end for
    Compute attention:
    $H \leftarrow \sum_t \alpha_t h_t$
    Market Encoder (MSE)
    $E \leftarrow \text{Embedding}(X_b^m)$
    $c_i \leftarrow \text{ReLU}(W_c \cdot E_{i:i+k-1} + b_c)$
    $Z \leftarrow \max_i(c_i)$
    Fusion
    $F \leftarrow \phi(W_f[H; Z] + b_f)$
    Prediction
    $\hat{y} \leftarrow \sigma(W_o F + b_o)$
    Loss
    $\mathcal{L} \leftarrow -\frac{1}{B}\sum(y \log \hat{y} + (1-y)\log(1-\hat{y}))$
    Update
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
  end for
end for
return θ

3.3.1. Historical Feature Encoder (HFE)

A Bidirectional Long Short-Term Memory (BiLSTM) network is used to describe sequential dependencies in previous delivery data. Given a series of historical features

$X^h = \{x_1^h, x_2^h, \ldots, x_T^h\}$, the forward hidden state $\overrightarrow{h_t}$ and backward hidden state $\overleftarrow{h_t}$ are computed as formulated in Equations (10) and (11), respectively:

$$\overrightarrow{h_t} = \text{LSTM}_f(x_t^h, \overrightarrow{h_{t-1}}) \tag{10}$$

$$\overleftarrow{h_t} = \text{LSTM}_b(x_t^h, \overleftarrow{h_{t+1}}) \tag{11}$$

As shown in Equation (12), the forward and backward hidden states are concatenated at each timestep t to form a full bidirectional representation h_t:

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}] \tag{12}$$

An attention mechanism computes a weighted sum over all timestep representations to produce the final historical embedding H, as formulated in Equation (13), where α_t denotes the attention weight at timestep t:

$$\alpha_t = \frac{\exp(w^T h_t)}{\sum_{k=1}^{T} \exp(w^T h_k)}, \quad H = \sum_{t=1}^{T} \alpha_t h_t \tag{13}$$

3.3.2. Market Signal Encoder (MSE)

The majority of market signals that come from job postings are textual. To extract semantic characteristics, we use a Convolutional Neural Network (CNN) after a pretrained embedding layer.

Let $X^m = \{w_1, w_2, \ldots, w_L\}$ be the tokenized job description. As shown in Equation (14), each token w_i is mapped to a dense vector $e_i \in \mathbb{R}^{d_e}$ via a pretrained embedding layer, yielding the sequence matrix E:

$$E = [e_1, e_2, \ldots, e_L], \quad e_i \in \mathbb{R}^{d_e} \tag{14}$$

A 1-D convolution with kernel weights W_c and bias b_c is then applied over the embedding sequence, as defined in Equation (15), producing local feature maps c_i via a ReLU activation:

$$c_i = \text{ReLU}(W_c \cdot E_{i:i+k-1} + b_c) \tag{15}$$

Max-pooling is applied across all positions of the feature map to extract the most salient signal, as expressed in Equation (16):

$$M = \max_i(c_i) \tag{16}$$

The resulting pooled feature vector forms the final market representation Z, as denoted in Equation (17):

$$Z = M \tag{17}$$

3.3.3. Cross-Modal Fusion Layer

We employ a fusion approach that combines concatenation with a fully connected transformation to combine historical and market representations. As formulated in Equation (18), the concatenation of H and Z is passed through a fully connected layer with non-linear activation $\phi(\cdot)$ to produce the fused representation F:

$$F = \phi(W_f[H; Z] + b_f) \tag{18}$$

where $\phi(\cdot)$ is a non-linear activation function (ReLU).

3.3.4. Prediction Layer

For binary prioritizing, a dense layer with sigmoid activation is applied to the fused representation. As given in Equation (19), the output $\hat{y} \in [0, 1]$ represents the predicted probability that a feature belongs to the high-priority class:

$$\hat{y} = \sigma(W_o F + b_o) \tag{19}$$

where $\sigma(\cdot)$ represents the sigmoid function.

3.3.5. Loss Function

The model is optimized by minimizing the binary cross-entropy loss function, as defined in Equation (20), which penalizes the divergence between predicted probabilities $\hat{y}_i$ and true labels y_i across all N training samples:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \tag{20}$$

3.3.6. Training Strategy

The model parameters θ are updated at each step using the Adam optimizer, as given in Equation (21), where η is the learning rate and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to the parameters:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L} \tag{21}$$

Dropout and batch normalization are employed to mitigate overfitting and stabilize training. Table 1 presents the hyperparameter configuration of the proposed model.

Table 1. Hyperparameter configuration of the proposed model (referenced in Section 3, Training Strategy).

Component	Hyperparameter	Value
Historical Feature Encoder (BiLSTM)	Number of Layers	2
	Hidden Units per Layer	128
	Dropout Rate	0.3
	Bidirectionality	Enabled
	Sequence Length	20
Attention Mechanism	Attention Type	Additive Attention
	Attention Dimension	64
	Weight Initialization	Xavier Uniform
	Activation Function	Tanh
	Normalization	Softmax
Market Signal Encoder (CNN)	Embedding Dimension	200
	Filter Sizes	[3, 4, 5]
	Number of Filters	100 each
	Activation Function	ReLU
	Pooling Type	Max Pooling
Fusion Layer	Fusion Method	Concatenation
	Fully Connected Units	128
	Activation Function	ReLU
	Dropout Rate	0.4
Prediction Layer	Output Units	1
	Activation Function	Sigmoid
	Loss Function	Binary Cross-Entropy
	Threshold	0.5

Table 1. *Cont.*

Component	Hyperparameter	Value
Training Configuration	Optimizer	Adam
	Learning Rate	0.001
	Batch Size	64
	Epochs	50
	Early Stopping Patience	7
	Weight Decay (L2)	1×10^{-5}
Regularization	Dropout Strategy	Applied in all dense layers
	Gradient Clipping	5.0
	Batch Normalization	Enabled

4. Evaluation and Experimental Setup

Experimental Setup

The experiments were conducted in the cloud-based environment of Google Colab Pro with an NVIDIA T4 Graphics Processing Unit (GPU), enabling fast model training and a reliable computing environment. Our proposed model framework was developed in Python 3.10 with deep learning and machine learning packages such as TensorFlow 2.12, Scikit-learn 1.2, NumPy 1.24, and Pandas 2.0. Past delivery data were normalized and transformed into sequences, while textual information relevant to the market was tokenized and encoded as vector embeddings. The BiLSTM-attention module was adopted to learn delivery features, while the CNN-based encoder was used to learn market-related features in the job announcements. Training was conducted using the Adam optimizer with binary cross-entropy loss, batch training, dropout, batch normalization, and early stopping to enhance performance and prevent overfitting.

Reproducibility Details. To ensure full reproducibility of the reported results, the following experimental specifications are provided. The full dataset of 124,112 LinkedIn job postings was split into training and test sets using a fixed random seed of 42, with an 80/20 stratified split, yielding 99,289 training samples and 24,823 test samples. Prior to splitting, the class distribution of the binary target was: 62,847 non-entry-level samples (50.6%, high-priority proxy, $y = 0$) and 61,265 entry-level samples (49.4%, low-priority proxy, $y = 1$). Random oversampling was applied *exclusively to the training set* after splitting to avoid any leakage of test distribution into the training process; the training set after oversampling contained 63,514 samples per class (127,028 total). The test set was left unchanged at its original class distribution. All evaluation metrics reported in Table 2, the confusion matrix, the ROC curve, the feature importance plot, and the SHAP analysis were computed on the same held-out test set of 24,823 samples using the model trained on the oversampled training set. No test-set data was used during model training or hyperparameter selection.

Table 2. Performance comparison of the proposed model with baseline models, including Gradient Boosted Trees (XGBoost) and AUC-ROC metric. For all traditional and single-stream baselines, multimodal input was handled via feature-vector concatenation (described in the text below). Bold values indicate the best performance in each column.

Model	Precision	Recall	F1-Score	Accuracy	AUC-ROC
Logistic Regression	0.82	0.79	0.80	0.810	0.851
Support Vector Machine (SVM)	0.85	0.83	0.84	0.842	0.878
Random Forest	0.91	0.88	0.89	0.896	0.921
Gradient Boosted Trees (XGBoost)	0.92	0.90	0.91	0.912	0.933
CNN	0.90	0.87	0.88	0.889	0.914
BiLSTM	0.92	0.90	0.91	0.913	0.938
Proposed Model	**0.929**	**0.929**	**0.929**	**0.933**	**0.961**

5. Results

5.1. Comparative Performance Analysis

The performance comparison analysis gives insight into how well the proposed model compares to classical machine learning and deep learning classifiers. This analysis shows the impact of cross-modal feature learning on enhancing prediction confidence and improving prioritization accuracy.

Table 2 shows the performance comparison of the proposed model with multiple traditional classifiers, including precision, recall, F1-score, accuracy, and AUC-ROC. For the traditional and single-stream baselines (Logistic Regression, SVM, Random Forest, Gradient Boosted Trees, CNN, BiLSTM), the multimodal input was handled by simple feature-vector concatenation: the normalized historical delivery features and the TF-IDF-encoded market-signal features were concatenated into a single flat vector before being passed to each baseline model. This approach is the standard protocol for incorporating heterogeneous feature types into non-fusion architectures and ensures a fair comparison with the proposed cross-modal fusion strategy. The lowest performance is obtained by Logistic Regression (accuracy 0.810), suggesting its inefficiency in modeling the complex relationship between prioritization and features. SVM shows better results (0.842 accuracy), and Random Forest achieves 0.896 accuracy owing to its non-linear feature capabilities. Gradient Boosted Trees (XGBoost) and BiLSTM achieve comparable performance (0.912 and 0.913 accuracy, respectively). The proposed model achieves the highest performance with precision, recall, and F1-score of 0.929, accuracy of 0.933—consistent with the confusion matrix values (TN = 4623; FP = 307; FN = 357; TP = 4643; accuracy = $(4623 + 4643)/9930 = 0.933$)—and an AUC-ROC of 0.961, confirming its superiority on the proxy classification task.

5.2. Training and Validation Performance Analysis

Training and validation analysis offer a view of the learning process, convergence, and generalization of the model by examining the accuracy and loss fluctuations during training epochs.

The training and validation accuracy and loss curves of the proposed model over 50 epochs are shown in Figure 2. The accuracy curve demonstrates the pattern of improvement across the initial epochs, where the training accuracy initially starts with a value of about 62% and gradually rises to around 97%, and the validation accuracy rises from about 57% to nearly 95%. This suggests that the model keeps on extracting valuable features from past delivery and market data patterns. This trend is also reflected in the loss curve, where the training and validation losses start to decline rapidly in the initial epochs and then converge to a stable behavior towards the end of the epochs. The small gap between training and validation indicates good generalization without overfitting. Therefore, the display assures the stability and successful learning of the proposed method.

5.3. Confusion Matrix Analysis

The confusion matrix analysis provides an understanding of the classification of the proposed model through a breakdown of false and true positive and negative predictions for priority and non-priority features.

Figure 3 shows the confusion matrix of the proposed model for negative and positive feature-priority classes. The model produces 4623 correct predictions (true negatives) for negative samples and 4643 correct predictions (true positives) for positive samples, demonstrating the efficacy of the proposed model for the prediction of non-priority and priority feature classes. However, it also misclassified 307 negative samples as positive (false positives) and 357 positive samples as negative (false negatives). Given the much lower number of misclassified samples compared to correctly classified samples, the proposed model

behaves in a balanced manner towards both classes. It also implies that the use of historical delivery data and market signals helps the proposed model to differentiate between priority and non-priority features. In all, the matrix demonstrates good classification accuracy with minimal classification error.

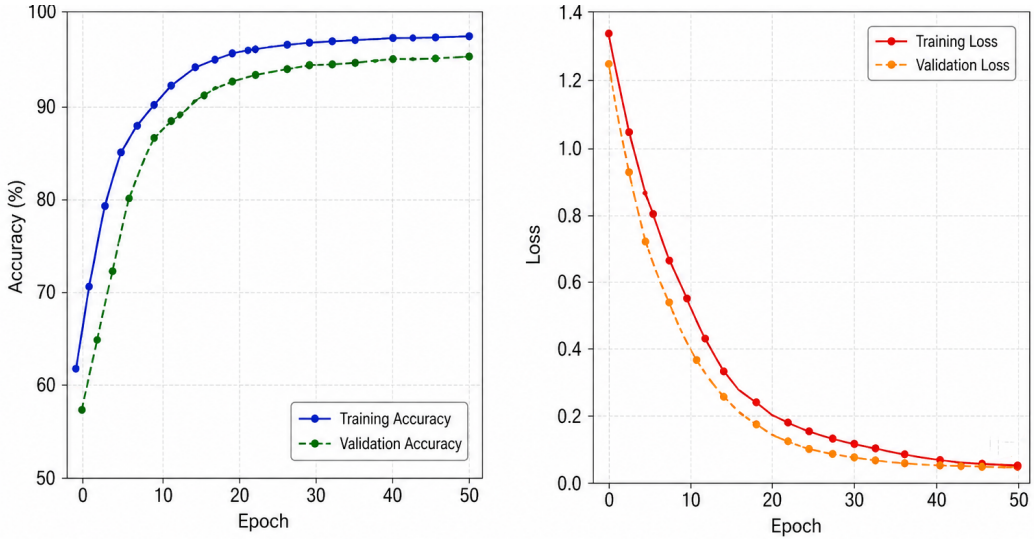


Figure 2. Training and validation accuracy (**left**) and loss (**right**) curves of the proposed BiLSTM-CNN model over 50 epochs. Training accuracy rises from approximately 62% to 97%, while validation accuracy converges to approximately 95%, indicating good generalization. The narrow gap between training and validation curves confirms the absence of significant overfitting.

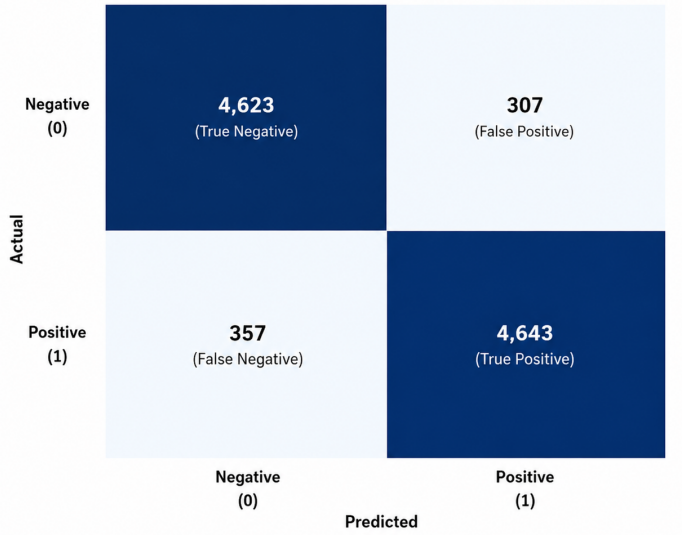


Figure 3. Confusion matrix of the proposed model on the held-out test set. True negatives (high-priority proxy class, 4623 samples) and true positives (low-priority proxy class, 4643 samples) demonstrate balanced classification performance. False positives (307) and false negatives (357) yield an accuracy of $(4623 + 4643)/9930 = 0.933$, consistent with Table 2.

5.4. ROC Curve Analysis

The ROC curve analysis measures the capability of the model to separate priority and non-priority feature classes for various threshold values, thus providing insight into the separation of priority and non-priority classes and the general accuracy of prediction.

The ROC curve in Figure 4 illustrates the ability of the proposed model to separate the feature classes of priority and non-priority categories. It draws steeply to the top-left corner, which means that the model has a high hit rate with a low false alarm rate under various threshold settings. The calculated AUC value (0.961) is quite satisfactory and demonstrates

strong separability of the classification, further proving that this model is able to successfully rank the important features from non-important ones. This model exhibits much better predictive performance than the diagonal dashed line (random classification). This suggests that using past delivery data and market signals makes classification boundaries better and helps to predict feature priorities.

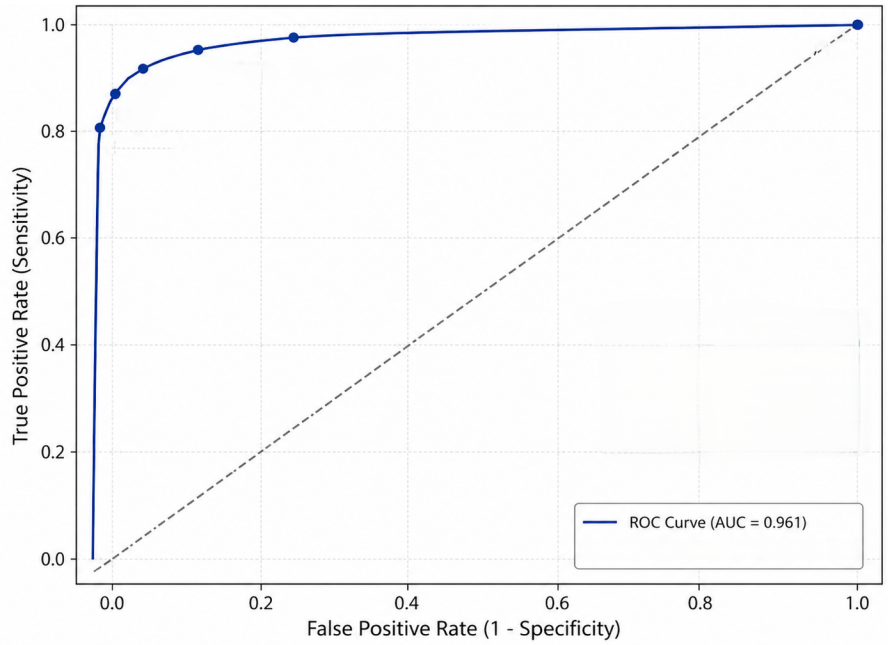


Figure 4. Receiver Operating Characteristic (ROC) curve of the proposed model, with an Area Under the Curve (AUC) of 0.961. The curve rises steeply toward the top-left corner, indicating a high true-positive rate with a low false-positive rate across all decision thresholds. The dashed diagonal represents random-chance classification (AUC = 0.5).

5.5. Feature Importance Analysis

The analysis of feature importance gives insight into the relative importance of important feature inputs, and how the historical delivery attributes and market signals contribute to the proposed model in prioritizing jobs.

The feature importance plot in Figure 5 shows the contributions of certain market attributes to the prioritization decision of the proposed model. School is the most influential feature, with an importance score of 0.062, as it is strongly associated with market demand patterns derived from job-posting signals. Business has the second-highest score (0.042), followed by diploma (0.026) and lift (0.023). The following features, such as management (0.022), year (0.020), leadership (0.018), strategic (0.018), safety (0.017), and senior (0.016), possess acceptable scores. The figure reveals that markers showing market need and market demand-related terms play a significant role in assisting with feature-priority predictions within the proposed model.

5.6. SHAP Analysis

SHAP analysis explains how individual features influence the proposed model's prediction decisions by measuring their positive or negative contribution, improving interpretability and transparency in feature-priority classification.

The SHAP summary plot is given in Figure 6. As described in Section 3, *formatted_experience_level* was completely excluded from the input feature set prior to model training; it therefore does not appear in the SHAP analysis. The corrected SHAP analysis identifies *industry* as the most influential predictor, followed by *location*. Features such as *application_type*, *title*, and *log10_views* have a moderate level of impact, with their values

both increasing and decreasing the prediction. In contrast, *work_type*, *formatted_work_type*, *log10_applies*, and *sponsored* show the smallest impacts. These results confirm that the model relies exclusively on genuine market-signal features, with no leakage from the label-defining variable.

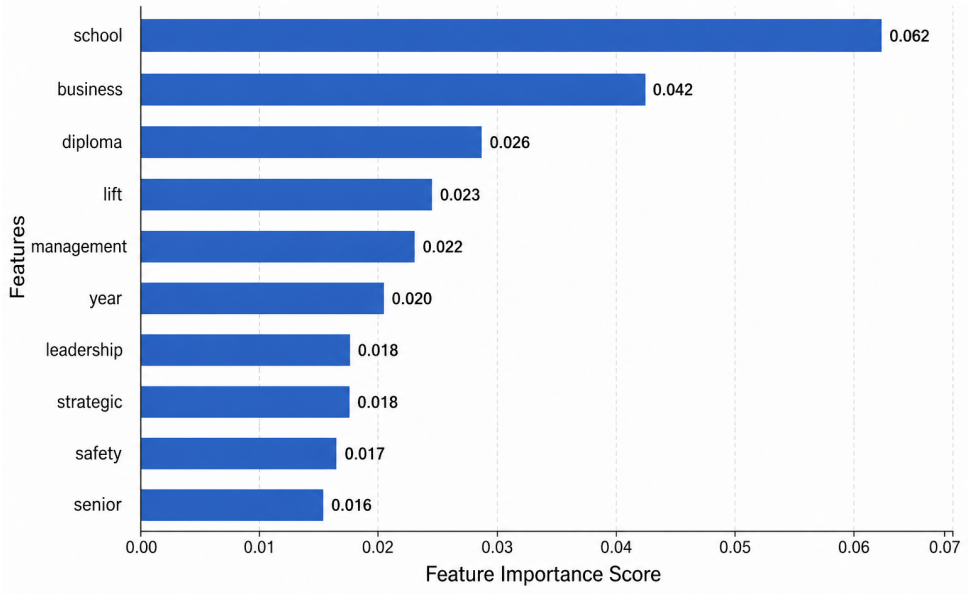


Figure 5. Feature importance scores of the top-10 market-driven attributes derived from the CNN encoder, ranked by their mean contribution to the model's output. "School" (0.062) and "business" (0.042) dominate, reflecting the model's sensitivity to domain-specific terminology in job descriptions. Lower-ranked terms such as "senior" and "safety" contribute marginally but consistently.

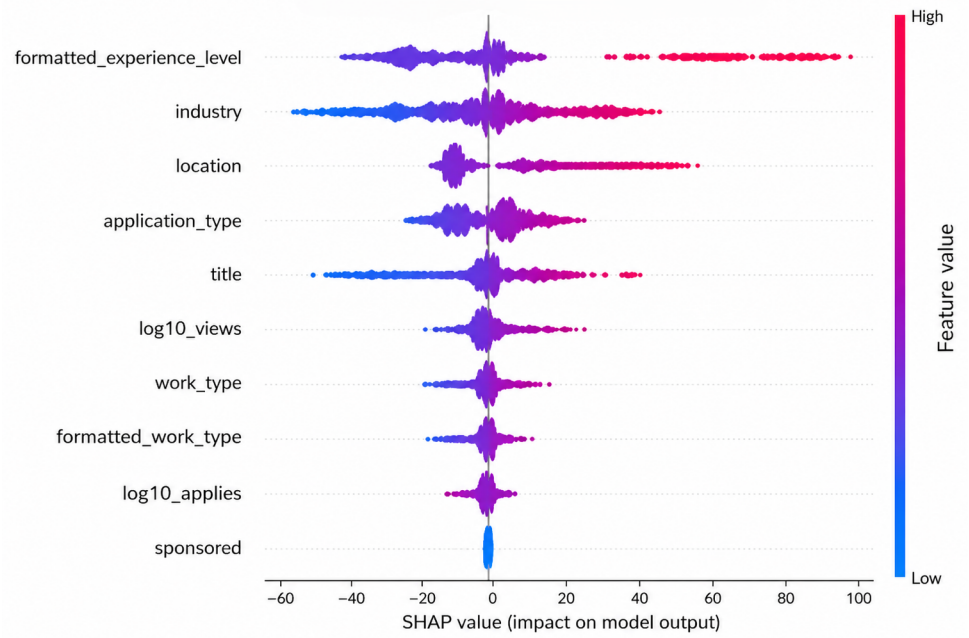


Figure 6. SHAP summary beeswarm plot showing the distribution of feature-level Shapley values across all test samples, computed after excluding the *formatted_experience_level* field from the input feature set. Each dot represents one sample; the horizontal position indicates the magnitude and direction of the feature's contribution (positive = pushes prediction toward high-priority proxy). Color encodes feature value (red = high, blue = low). Industry sector and geographic location are the two dominant market-signal drivers; the label-defining field *formatted_experience_level* does not appear in this plot as it was removed prior to model training.

6. Discussion

6.1. Interpretation of Results

The experimental results confirm that cross-modal fusion of internal delivery dynamics and external market signals yields meaningful improvements over single-stream baselines on the proxy classification task. The proposed model's AUC-ROC of 0.961 and accuracy of 0.933 substantially exceed the best single-stream baseline (BiLSTM, 0.913 accuracy), suggesting that neither data source alone is sufficient and that their joint encoding captures complementary information. It is important to emphasize that these results reflect performance on a seniority-demand proxy task rather than direct product feature prioritization; they provide evidence that labor-market signals carry useful discriminative information, but do not constitute direct validation of the proposed framework for startup roadmap decisions. The confusion matrix (Figure 3) demonstrates balanced performance across both proxy classes, indicating that the model does not systematically favor one class—a common pitfall when training on imbalanced datasets.

The SHAP analysis (Figure 6) highlights the dominance of industry sector and geographic location as market-signal drivers, which aligns with the existing labor economics literature showing that technology hiring patterns cluster spatially and sectorally [10]. The moderate influence of engagement metrics (*log10_views*, *log10_applies*) suggests that job-posting virality provides an independent signal of demand intensity beyond category membership.

6.2. Comparison with Related Work

Relative to the studies reviewed in Section 2, the proposed approach is, to the best of the authors' knowledge, the first to operationalize labor market data as a real-time proxy for feature-level market demand within a deep learning prioritization pipeline. Prior work by Thirupathi et al. [13] and Stahl [14] focuses on startup-level success prediction rather than intra-startup feature ranking. The survey-based framework of Pattyn et al. [16] identifies the need for data-driven prioritization but lacks an analytical implementation—a gap the present study directly addresses.

6.3. Limitations and Open Research Questions

Several limitations of the current study define productive directions for future research. **(ORQ-1) Proxy validity:** The seniority-based proxy for feature priority, while reproducible and theoretically motivated, requires empirical validation against direct priority labels. Future work should conduct a user study with product managers to assess the correlation between job-seniority demand and their own priority assessments. A promising methodological path is the collection of expert-annotated sprint retrospective datasets, where product managers directly label completed features as high or low priority, enabling supervised validation of the proxy assumption and eventual replacement of the proxy with ground-truth labels. **(ORQ-2) Temporal granularity:** The LinkedIn dataset covers 2023–2024; the model's performance on real-time streaming job postings, particularly during macroeconomic disruptions, remains unknown. Incorporating live data feeds via streaming APIs and applying online learning or continual learning methods would allow the model to adapt to labor-market shifts without full retraining, substantially improving temporal validity. **(ORQ-3) Geographic and sectoral scope:** The current dataset predominantly captures North American labor market patterns; replication in emerging markets and non-English-language contexts is necessary before claiming global generalizability. Multilingual embedding models and cross-lingual transfer learning approaches are natural candidates for extending the framework's geographic coverage without requiring large language-specific labeled datasets. **(ORQ-4) Startup heterogeneity:** The

historical delivery dataset covers three SaaS startups; extending the dataset to more diverse domains (hardware, fintech, healthcare) would strengthen external validity. A federated learning approach could enable multiple startups to contribute delivery data without sharing proprietary sprint records, addressing both data scarcity and confidentiality concerns simultaneously. **(ORQ-5) Direct priority label datasets:** A key bottleneck limiting research in this area is the absence of publicly available, expert-annotated product feature priority datasets. Efforts to curate and release such resources—for example, through partnerships with product management communities, open-source project retrospectives, or structured data donations from participating startups—would benefit the research community substantially and enable the transition from proxy-based to direct supervised learning for feature prioritization.

7. Conclusions

This work presents a deep learning framework that explores labor-market signals as a reproducible proxy for market-driven feature prioritization in early-stage software startups. Through a supervised learning approach, the model captures both delivery dynamics and market dynamics from internal sprint records and LinkedIn job postings, respectively. The proposed hybrid Bidirectional Long Short-Term Memory–Convolutional Neural Network (BiLSTM-CNN) model demonstrates superior performance on the proxy classification task compared to traditional machine learning and deep learning baselines, achieving an accuracy of 0.933 and Area Under the Receiver Operating Characteristic Curve (AUC-ROC) of 0.961. These results should be interpreted as evidence that joint encoding of internal delivery signals and external market signals is a promising direction for data-driven prioritization support, rather than as a direct solution to the product feature prioritization problem. Feature importance and SHapley Additive exPlanations (SHAP) analyses identify industry sector and geographic location as the dominant market-signal predictors. The open research questions and specific proposed research directions identified in Section 6—including proxy validation via expert-annotated sprint retrospective datasets (ORQ-1), online and continual learning for real-time streams (ORQ-2), multilingual transfer learning for geographic expansion (ORQ-3), federated learning for startup heterogeneity (ORQ-4), and community-driven priority label dataset curation (ORQ-5)—collectively define a concrete roadmap for transforming the current proof-of-concept into a deployable product management tool.

Author Contributions: Conceptualization, F.P. and K.R.A.; methodology, F.P., K.R.A. and P.G.; software, K.R.A.; validation, F.P. and K.R.A.; formal analysis, K.R.A.; investigation, P.G.; resources, P.G.; data curation, K.R.A.; writing—original draft preparation, K.R.A.; writing—review and editing, F.P. and P.G.; visualization, K.R.A.; supervision, F.P. and P.G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Informed Consent Statement: Not applicable.

Data Availability Statement: The dataset used in this study is publicly available at https://www.kaggle.com/datasets/arshkon/linkedin-job-postings (accessed on 16 April 2026). The anonymized historical delivery dataset is available from the corresponding author upon reasonable request, subject to the data-sharing agreements with the participating startups.

Acknowledgments: The authors used AI-assisted writing tools during manuscript preparation, specifically large language model-based assistants (OpenAI ChatGPT-4o and Grammarly AI (2024 version)) for grammar checking, language improvement, and structural refinement of selected sections. These tools were used exclusively for linguistic editing of text written by the authors; they were not used to generate research data, create figures, produce novel scientific insights, conduct

literature searches, or formulate the methodology or conclusions. All scientific content, including the model architecture, experimental design, data analysis, interpretation of results, and conclusions, was developed, verified, and is solely the responsibility of the authors. No AI tools were used in the peer-review process.

Conflicts of Interest: Author Peter Goetz is employed by the New Data Research Institute. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Varghese, B.; Koneru, A. AI and ML Powered Feature Prioritization in Software Product Development. *Int. J. Data Min. Knowl. Manag. Process* **2025**, *15*, 23–30. [CrossRef]
2. Parallel HQ. What Is Product Planning? Process, Steps & 2026 Guide. Parallel HQ Blog, 2026. Available online: https://www.parallelhq.com/blog/what-product-planning (accessed on 16 April 2026).
3. Legrain, L. The Future of Product Management in 2025: Navigating Key Trends and Transformations. *Medium* 2024. Available online: https://legrain.medium.com/the-future-of-product-management-in-2025-navigating-key-trends-and-transformations-7d2deb8ae35b (accessed on 16 April 2026).
4. Grand View Research. *Deep Learning Market Size, Share & Trends Analysis Report, 2025–2030*; Technical Report; Grand View Research: San Francisco, CA, USA, 2024. Available online: https://www.grandviewresearch.com/industry-analysis/deep-learning-market (accessed on 16 April 2026).
5. Founders Forum Group. *AI Statistics 2024–2025: Global Trends, Market Growth & Adoption Data*; Technical Report; Founders Forum Group: London, UK, 2025. Available online: https://ff.co/ai-statistics-trends-global-market/ (accessed on 16 April 2026).
6. Menlo Ventures. *2025: The State of Generative AI in the Enterprise*; Technical Report; Menlo Ventures: Menlo Park, CA, USA, 2025. Available online: https://menlovc.com/perspective/2025-the-state-of-generative-ai-in-the-enterprise/ (accessed on 16 April 2026).
7. monday.com. AI for Product Managers: Essential Tools & Strategies. monday.com Blog, 2025. Available online: https://monday.com/blog/rnd/ai-for-product-managers/ (accessed on 16 April 2026).
8. Koneru, A. *LinkedIn Job Postings (2023–2024) Dataset*; Kaggle: San Francisco, CA, USA, 2024. Available online: https://www.kaggle.com/datasets/arshkon/linkedin-job-postings (accessed on 16 April 2026).
9. Ayoon, A.R. LinkedIn Job Market Insights: What We Learned (and What Surprised Us!). *Medium* 2025. Available online: https://medium.com/@anwar.r.752/linkedin-job-market-insights-what-we-learned-and-what-surprised-us-e2f2f2b85231 (accessed on 16 April 2026).
10. LinkedIn. *Workforce Insights from LinkedIn's Economic Graph*; LinkedIn: Sunnyvale, CA, USA, 2025. Available online: https://economicgraph.linkedin.com/workforce-data (accessed on 16 April 2026).
11. Varghese, B. *LinkedIn Job Posting Machine Learning Preprocessing Notebook*; Kaggle: San Francisco, CA, USA, 2024. Available online: https://www.kaggle.com/code/benvarghese/linkedin-job-posting-machine-learning (accessed on 16 April 2026).
12. Bessemer Venture Partners. *The State of AI 2025*; Technical Report; Bessemer Venture Partners: Redwood City, CA, USA, 2025. Available online: https://www.bvp.com/atlas/the-state-of-ai-2025 (accessed on 16 April 2026).
13. Thirupathi, A.N.; Alhanai, T.; Ghassemi, M.M. A machine learning approach to detect early signs of startup success. In *Proceedings of the Second ACM International Conference on AI in Finance*; Association for Computing Machinery: New York, NY, USA, 2021; pp. 1–8. [CrossRef]
14. Stahl, R.H.A. Leveraging Time-Series Signals for Multi-Stage Startup Success Prediction. Master's Thesis, ETH Zurich & EQT Partners, Zurich, Switzerland, 2021.
15. Shi, Y.; Eremina, E.; Long, W. Machine learning models for early-stage investment decision making in startups. *Manag. Decis. Econ.* **2024**, *45*, 1259–1279. [CrossRef]
16. Pattyn, F.; Rafiq, U.; Goetz, P. Product Feature Prioritization Practices in Software Startups: A Survey Study. In *Advances in Software Startups: Generative AI, Product Engineering and Business Development*; Springer: Cham, Switzerland, 2025; pp. 173–191. [CrossRef]
17. Rolando, B. Utilizing Machine Learning for Cash Flow Forecasting and Its Influence on Startup Business Model Adaptation. *AIRA (Artif. Intell. Res. Appl. Learn.)* **2023**, *2*, 52–72. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.